40.1 ACP 协议概述

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~320k tokens，输出 ~6k tokens（本节）

随着 AI Agent 生态的快速扩展，不同 Agent 系统之间的互操作成为关键需求。ACP（Agent Communication Protocol，Agent 通信协议）应运而生——它定义了一套标准的 Agent 间通信协议，允许 IDE、编辑器和其他客户端以统一的方式与 AI Agent 交互。OpenClaw 通过 `src/acp/` 模块实现了 ACP 协议的服务端适配，将其作为连接外部客户端（如 IDE 插件）的桥梁。

> **衍生解释：Agent Communication Protocol (ACP)**
> ACP 是一种开放的 Agent 通信标准，类似于 LSP（Language Server Protocol）之于编程语言工具链的角色。LSP 让任何编辑器都能复用同一个语言服务器的能力（代码补全、跳转定义等），ACP 则让任何客户端都能与 AI Agent 进行标准化的交互——创建会话、发送提示、接收流式响应、管理工具调用等。ACP 使用 JSON-RPC 风格的 NDJSON（Newline-Delimited JSON）作为传输格式，通过 stdio 管道通信。

40.1.1 ACP SDK 依赖

OpenClaw 的 ACP 实现基于 `@agentclientprotocol/sdk` 包，该 SDK 提供了协议的核心类型和连接管理：

```
import {
  AgentSideConnection,  // Agent 端连接（服务端）
  ClientSideConnection, // 客户端连接
  ndJsonStream,         // NDJSON 流传输
  PROTOCOL_VERSION,     // 协议版本号
  type Agent,           // Agent 接口
  type PromptRequest,   // 提示请求
  type PromptResponse,  // 提示响应
  type SessionNotification, // 会话通知
  // ...
} from "@agentclientprotocol/sdk";
```

SDK 提供的关键抽象：

类型	角色	说明
`AgentSideConnection`	服务端	Agent 侧的连接管理，接收客户端请求
`ClientSideConnection`	客户端	客户端侧的连接管理，发送请求给 Agent
`ndJsonStream`	传输层	将 stdin/stdout 封装为 NDJSON 双向流
`Agent`	接口	Agent 必须实现的方法集合

40.1.2 ACP 服务端架构

OpenClaw 的 ACP 服务端由三层组成：

```
ACP 客户端（IDE / 终端）
    ↕ stdio（NDJSON）
AcpGatewayAgent（翻译层）
    ↕ WebSocket
OpenClaw Gateway
```

服务端启动

`serveAcpGateway()` 是 ACP 服务端的入口函数：

```typescript
// src/acp/server.ts（简化）

export function serveAcpGateway(opts: AcpServerOptions = {}) {
  const cfg = loadConfig();
  const connection = buildGatewayConnectionDetails({ config: cfg });
  const auth = resolveGatewayAuth({ ... });

  // 1. 创建 Gateway WebSocket 客户端
  const gateway = new GatewayClient({
    url: connection.url,
    token: token || undefined,
    password: password || undefined,
    clientName: "CLI",
    clientDisplayName: "ACP",
    onEvent: (evt) => agent?.handleGatewayEvent(evt),
    onHelloOk: () => agent?.handleGatewayReconnect(),
    onClose: (code, reason) =>
      agent?.handleGatewayDisconnect(`${code}: ${reason}`),
  });

  // 2. 将 stdin/stdout 封装为 NDJSON 流
  const input = Writable.toWeb(process.stdout);
  const output = Readable.toWeb(process.stdin);
  const stream = ndJsonStream(input, output);

  // 3. 创建 ACP Agent 连接
  new AgentSideConnection((conn) => {
    agent = new AcpGatewayAgent(conn, gateway, opts);
    agent.start();
    return agent;
  }, stream);

  gateway.start();
}
```

注意 stdio 方向的处理：ACP 协议中，Agent 通过 `stdout` **发送**消息给客户端，通过 `stdin` **接收**来自客户端的消息——因此 `input = stdout` 、 `output = stdin` 的赋值是正确的（从流的角度，写入 stdout 是 Agent 的输出，读取 stdin 是 Agent 的输入）。

命令行参数

ACP 服务端通过 `openclaw acp` 命令启动，支持以下参数：

```
openclaw acp [options]

Options:
  --url <url>              Gateway WebSocket URL
  --token <token>          Gateway 认证令牌
  --password <password>    Gateway 认证密码
  --session <key>          默认会话键
  --session-label <label> 默认会话标签
  --require-existing       仅使用已存在的会话
  --reset-session          首次使用前重置会话
  --no-prefix-cwd          不在提示前添加工作目录
  --verbose, -v            详细日志输出到 stderr
```

ACP Agent 信息

每个 ACP Agent 需要声明自己的身份信息：

```
export const ACP_AGENT_INFO = {
  name: "openclaw-acp",
  title: "OpenClaw ACP Gateway",
  version: VERSION,  // 动态读取 OpenClaw 版本号
};
```

40.1.3 翻译层：AcpGatewayAgent

`AcpGatewayAgent` 是整个 ACP 实现的核心——它实现了 ACP 的 `Agent` 接口，将 ACP 协议的请求翻译为 OpenClaw Gateway 的 RPC 调用，并将 Gateway 的事件流翻译回 ACP 的会话通知。

协议初始化

```
async initialize(_params: InitializeRequest): Promise<InitializeResponse> {
  return {
    protocolVersion: PROTOCOL_VERSION,
    agentCapabilities: {
      loadSession: true,
      promptCapabilities: {
        image: true,     // 支持图像输入
        audio: false,    // 不支持音频
        embeddedContext: true,  // 支持嵌入上下文
      },
      mcpCapabilities: {
        http: false,     // 不支持 MCP over HTTP
        sse: false,      // 不支持 MCP over SSE
      },
      sessionCapabilities: { list: {} },
    },
    agentInfo: ACP_AGENT_INFO,
    authMethods: [],
  };
}
```

初始化响应声明了 Agent 的能力集合。当前实现支持图像和嵌入上下文，但不支持音频和 MCP 服务器集成。

会话管理

ACP 的会话与 OpenClaw 的会话之间需要映射：

```
// 创建新会话
async newSession(params: NewSessionRequest): Promise<NewSessionResponse> {
  const sessionId = randomUUID();
  // 解析会话元数据（sessionKey, label, reset 等）
  const meta = parseSessionMeta(params._meta);
  // 解析最终的 sessionKey
  const sessionKey = await resolveSessionKey({
    meta,
    fallbackKey: `acp:${sessionId}`,
    gateway: this.gateway,
    opts: this.opts,
  });
  // 可选：重置会话
  await resetSessionIfNeeded({ meta, sessionKey, ... });
  // 创建 ACP 会话记录
  const session = this.sessionStore.createSession({
    sessionId,
    sessionKey,
    cwd: params.cwd,
  });
  // 发送可用命令列表
  await this.sendAvailableCommands(session.sessionId);
  return { sessionId: session.sessionId };
}
```

会话键的解析支持多种方式：

1. **通过标签**： `sessionLabel` → 调用 `sessions.resolve` RPC 按标签查找
2. **通过键**： `sessionKey` → 直接使用或验证存在性
3. **回退**：生成 `acp:{uuid}` 格式的新键

ACP 会话存储（`AcpSessionStore`）是一个简单的内存 Map，维护 `sessionId → AcpSession` 的映射，以及 `runId → sessionId` 的反向索引。

提示处理

```
async prompt(params: PromptRequest): Promise<PromptResponse> {
  const session = this.sessionStore.getSession(params.sessionId);
  // 取消之前的活跃运行
  if (session.abortController) {
    this.sessionStore.cancelActiveRun(params.sessionId);
  }

  // 提取文本和附件
  const userText = extractTextFromPrompt(params.prompt);
  const attachments = extractAttachmentsFromPrompt(params.prompt);
  // 可选：前缀工作目录
  const message = prefixCwd
    ? `[Working directory: ${session.cwd}]\n\n${userText}`
    : userText;

  // 转发到 Gateway 的 chat.send
  return new Promise<PromptResponse>((resolve, reject) => {
    this.pendingPrompts.set(params.sessionId, {
      sessionId, sessionKey, idempotencyKey: runId,
      resolve, reject,
    });

    this.gateway.request("chat.send", {
      sessionKey: session.sessionKey,
      message,
      attachments: attachments.length > 0 ? attachments : undefined,
      idempotencyKey: runId,
      // ...
    }, { expectFinal: true });
  });
}
```

提示处理的关键设计：

- **工作目录前缀**：默认在用户消息前添加 `[Working directory: /path/to/project]` ，让 Agent 知道用户的工作上下文
- **幂等键**：使用 `runId` （UUID）作为 `idempotencyKey` ，防止重复处理
- **Promise 桥接**：将 Gateway 的事件驱动响应转换为 Promise 风格的同步返回

事件翻译

Gateway 返回的事件需要翻译为 ACP 的会话更新通知：

```
// chat 事件 → ACP 会话更新
private async handleChatEvent(evt: EventFrame): Promise<void> {
  const { sessionKey, state, messageData } = evt.payload;
  const pending = this.findPendingBySessionKey(sessionKey);

  if (state === "delta" && messageData) {
    // 增量文本 → agent_message_chunk
    const fullText = /* 提取文本 */;
    const newText = fullText.slice(pending.sentTextLength);
    pending.sentTextLength = fullText.length;
    await this.connection.sessionUpdate({
      sessionId: pending.sessionId,
      update: {
        sessionUpdate: "agent_message_chunk",
        content: { type: "text", text: newText },
      },
    });
    return;
  }

  if (state === "final") {
    // 完成 → end_turn
    this.finishPrompt(pending.sessionId, pending, "end_turn");
  }
}

// agent 事件 → 工具调用通知
private async handleAgentEvent(evt: EventFrame): Promise<void> {
  if (stream === "tool" && phase === "start") {
    await this.connection.sessionUpdate({
      sessionId: pending.sessionId,
      update: {
        sessionUpdate: "tool_call",
        toolCallId,
        title: formatToolTitle(name, args),
        status: "in_progress",
        kind: inferToolKind(name),
      },
    });
  }
}
```

事件翻译的两个核心映射：

Gateway 事件	ACP 通知	说明
`chat.delta`	`agent_message_chunk`	流式文本增量（差分计算：`fullText - sentSoFar`）
`chat.final`	`PromptResponse(end_turn)`	提示完成
`chat.aborted`	`PromptResponse(cancelled)`	被用户取消
`chat.error`	`PromptResponse(refusal)`	Agent 拒绝或错误
`agent.tool.start`	`tool_call`	工具调用开始
`agent.tool.result`	`tool_call_update`	工具调用完成/失败

增量文本的处理值得注意：Gateway 的 `delta` 事件发送的是**累积全文**（非增量），ACP 翻译层通过 `sentTextLength` 计数器追踪已发送的字符数，只发送新增部分。这种设计的优势是支持乱序和重传——即使错过一个 delta 事件，后续的完整文本也能自我纠正。

工具类型推断

`inferToolKind()` 根据工具名称推断其 ACP 工具类型：

```
export function inferToolKind(name?: string): ToolKind {
  const normalized = name?.toLowerCase();
  if (normalized?.includes("read"))   return "read";
  if (normalized?.includes("write"))  return "edit";
  if (normalized?.includes("delete")) return "delete";
  if (normalized?.includes("exec"))   return "execute";
  if (normalized?.includes("fetch"))  return "fetch";
  if (normalized?.includes("search")) return "search";
  return "other";
}
```

这是一种**启发式**分类——通过关键词匹配将 OpenClaw 丰富的工具名称映射到 ACP 定义的有限工具类别中。

40.1.4 ACP 客户端

`client.ts` 提供了 ACP 客户端的参考实现，用于从命令行与 ACP Agent 交互：

```typescript
export async function createAcpClient(opts: AcpClientOptions = {}) {
  // 1. 启动 ACP 服务端进程
  const agent = spawn("openclaw", ["acp", ...serverArgs], {
    stdio: ["pipe", "pipe", "inherit"],
    cwd,
  });

  // 2. 建立 NDJSON 流连接
  const stream = ndJsonStream(
    Writable.toWeb(agent.stdin),
    Readable.toWeb(agent.stdout),
  );

  // 3. 创建客户端连接
  const client = new ClientSideConnection(
    () => ({
      sessionUpdate: async (params) => {
        // 处理流式输出
        printSessionUpdate(params);
      },
      requestPermission: async (params) => {
        // 自动批准权限请求
        return { outcome: { outcome: "selected", optionId: "allow" } };
      },
    }),
    stream,
  );

  // 4. 协议初始化
  await client.initialize({
    protocolVersion: PROTOCOL_VERSION,
    clientCapabilities: {
      fs: { readTextFile: true, writeTextFile: true },
      terminal: true,
    },
    clientInfo: { name: "openclaw-acp-client", version: "1.0.0" },
  });

  // 5. 创建会话
  const session = await client.newSession({ cwd, mcpServers: [] });
  return { client, agent, sessionId: session.sessionId };
}
```

客户端通过 `stdio` 与 ACP 服务端通信：

```
[ACP 客户端进程]
    stdin  → 写入请求  → [ACP 服务端的 stdin]
    stdout ← 接收响应  ← [ACP 服务端的 stdout]
    stderr ← 日志输出  ← [ACP 服务端的 stderr]
```

交互式模式（`runAcpClientInteractive`）提供了一个简单的 REPL 界面：

```
OpenClaw ACP client
Session: a1b2c3d4-...
Type a prompt, or "exit" to quit.

> 帮我分析一下这个项目的架构
[tool] read: filePath: src/index.ts (in_progress)
[tool update] tool_123: completed
这个项目采用了分层架构...

[end_turn]

>
```

会话更新处理

客户端接收的会话更新通知被翻译为人类可读的输出：

```
function printSessionUpdate(notification: SessionNotification) {
  switch (notification.update.sessionUpdate) {
    case "agent_message_chunk":
      // 流式输出 Agent 的文本回复
      process.stdout.write(notification.update.content.text);
      break;
    case "tool_call":
      // 工具调用开始
      console.log(`\n[tool] ${update.title} (${update.status})`);
      break;
    case "tool_call_update":
      // 工具调用状态更新
      console.log(`[tool update] ${update.toolCallId}: ${update.status}`);
      break;
    case "available_commands_update":
      // 可用命令列表
      const names = update.availableCommands.map(cmd => `/${cmd.name}`);
      console.log(`\n[commands] ${names.join(" ")}`);
      break;
  }
}
```

40.1.5 可用命令

ACP 服务端在会话创建时推送可用的斜杠命令列表：

```
export function getAvailableCommands(): AvailableCommand[] {
  return [
    { name: "help", description: "Show help and common commands." },
    { name: "status", description: "Show current status." },
    { name: "think", description: "Set thinking level." },
    { name: "model", description: "Select a model." },
    { name: "reset", description: "Reset the session." },
    { name: "compact", description: "Compact the session history." },
    { name: "stop", description: "Stop the current run." },
    // ... 共 25+ 个命令
  ];
}
```

这些命令与 OpenClaw 的聊天命令系统（第 25 章）一致，但通过 ACP 协议暴露给外部客户端，使 IDE 插件等工具能够提供命令补全和快捷操作。

40.1.6 元数据辅助函数

ACP 协议的 `_meta` 字段用于传递扩展信息。 `meta.ts` 提供了类型安全的读取工具：

```
// 多键名查找，兼容不同客户端的命名习惯
export function readString(
  meta: Record<string, unknown> | null | undefined,
  keys: string[],
): string | undefined {
  for (const key of keys) {
    const value = meta[key];
    if (typeof value === "string" && value.trim()) {
      return value.trim();
    }
  }
  return undefined;
}
```

多键名查找设计使得同一配置可以通过不同的键名传递。例如， `readString(meta, ["sessionKey", "session", "key"])` 接受三种键名——这提高了与不同客户端实现的兼容性。

本节小结

1. **ACP 是 Agent 间通信的标准协议**，类似于 LSP 之于语言工具链，通过 NDJSON over stdio 进行通信。
2. **三层架构**：ACP 客户端 ↔ AcpGatewayAgent（翻译层）↔ OpenClaw Gateway，翻译层负责协议转换。
3. **AcpGatewayAgent** 实现了 ACP 的 `Agent` 接口，将 `prompt` / `newSession` / `cancel` 等请求翻译为 Gateway 的 `chat.send` / `sessions.resolve` / `chat.abort` RPC。
4. **事件翻译** 将 Gateway 的 `chat.delta` / `agent.tool` 事件转换为 ACP 的 `agent_message_chunk` / `tool_call` 通知，增量文本通过差分计算避免重复发送。
5. **会话映射** 支持通过键、标签或自动生成三种方式建立 ACP 会话与 OpenClaw 会话的对应关系。
6. **工具类型推断** 使用启发式关键词匹配将 OpenClaw 工具名映射到 ACP 标准类别。

7. **命令暴露** 将 25+ 个斜杠命令通过 ACP 协议暴露给外部客户端，支持 IDE 插件的命令补全。